

# Making Program Refactoring Safer

**Gustavo Soares, Rohit Gheyi, Dalton Serey, and Tiago Massoni,**  
*Federal University of Campina Grande*

A tool for improving safety during refactoring automatically analyzes the transformation and generates a test suite suited for detecting behavioral changes.

**R**efactoring changes a software system in such a way that it doesn't alter the code's external behavior but improves its internal structure.<sup>1,2</sup> Developers perform refactoring either manually, which is error-prone and time-consuming, or with the help of IDEs such as Eclipse, Netbeans, JBuilder, and IntelliJ, which can support refactoring.

In general, each refactoring can contain several preconditions to preserve the observable behavior. For instance, to rename an attribute, name conflicts can't be present. However, refactoring tools typically don't implement all preconditions because formally establishing them is nontrivial. Therefore, refactoring tools often allow wrong transformations to be applied without any warning. Figure 1 shows a transformation example with subtle changes in program behavior when applying it through the Eclipse 3.4.2 IDE.<sup>3</sup> Figure 1a shows a program containing class A and its subclass B. The method `test` yields 10. When we apply the pull-up refactoring to the method `k(int)` using Eclipse, we get the resulting code presented in Figure 1b. The method `test` in the target program yields 20, instead of 10. Therefore, the transformation doesn't preserve behavior if we use the Eclipse 3.4.2 IDE.

The current practice to avoid behavioral changes in refactorings relies on solid tests (see the sidebar "Related Work in Automated Testing Refactoring Tools").<sup>1</sup> However, test suites often don't catch behavioral changes during transformations. The tools might refactor them as well (for instance, with the `rename` method) because they rely on the program structure the refactor-

ing modifies.<sup>2</sup> In this case, the tool changes the method invocations on the test suite, and checks the original and refactored programs against different test suites. Clearly, this scenario is undesirable because the refactoring tool can change the test suite meaning.<sup>4</sup>

Here, we describe and evaluate SafeRefactor, a tool for checking refactoring safety in sequential Java programs using Eclipse IDE. For each transformation, it analyzes the transformation and generates a test suite for detecting behavioral changes.

## SafeRefactor

SafeRefactor is an Eclipse plugin ([www.dsc.ufcg.edu.br/~spg/saferefactor](http://www.dsc.ufcg.edu.br/~spg/saferefactor)) that receives a source code and a refactoring to apply (input) from the developer. It then reports whether it's safe to apply the transformation (output).

Suppose we use SafeRefactor in the program in Figure 1a. The whole process has seven sequential steps for each refactoring application (see Figure 2). First, the developer selects the refactoring to be applied on the source program (step 1.1) and then uses SafeRefactor (step 1.2) to check the refactoring safety (steps 2-7).

SafeRefactor generates a target program based

on the desired transformation using the Eclipse refactoring API (step 2). In step 3, static analysis automatically identifies methods common in both source and target programs. Step 4 generates unit tests for methods identified in step 3. In step 5, the plugin runs the generated test suite on the source program and then runs the same test suite on the target program (step 6). If a test passes in one of the programs and fails in the other, the plugin detects a behavioral change and reports it to the user (step 7). Otherwise, the developer can have more confidence that the transformation doesn't introduce behavioral changes.

The static analysis goal (step 3) is to identify methods in common that have exactly the same modifier, return type, qualified name, parameter types, and exceptions thrown in source and target programs. For example, Figure 1a and 1b have `A.k(long)` and `B.test()` in common.

After identifying a set of useful methods, the plugin uses Randoop to generate unit tests (step 4) for classes within a time limit.<sup>5</sup> A unit test typically consists of a sequence of method and constructor invocations that creates and mutates objects with random values, plus a JUnit assertion. Randoop executes the program to receive feedback gathered from executing test inputs as they're created, to avoid generating redundant and illegal inputs.<sup>5</sup> It creates method sequences incrementally by randomly selecting a method call to apply and selecting arguments from previously constructed sequences. Each sequence is executed and checked against a set of contracts. For instance, an object must be equal to itself. Our tool uses the Randoop default contracts—we modified Randoop to remove some defects and pass a set of methods as a parameter. All tests generated only contain calls to the methods step 3 identifies. The default time limit is two seconds. Steps 3 and 4 ensure that the same tests can run on the source and target programs.

The whole process to finish Figure 1 takes less than eight seconds on a dual-processor 2.2-GHz Dell Vostro 1400 laptop with 2 Gbytes RAM, and generates 154 unit tests (151 of them failed in the target program). In this case, SafeRefactor reports that the refactoring shouldn't be applied. In other situations, SafeRefactor can report compilation errors that refactoring tools might introduce. If SafeRefactor doesn't find a behavior change or compilation error, it reports that the transformation is most likely sound.

In William Opdyke's thesis, he compares the observable behavior of two programs with respect to the `main` method (a method in common).

## Related Work in Automated Testing Refactoring Tools

Brett Daniel and his colleagues proposed a technique for automated testing refactoring tools.<sup>1</sup> They found some compilation errors introduced by Eclipse and Netbeans. Friedrich Steimann and Andreas Thies<sup>2</sup> and Torbjörn Ekman and his colleagues<sup>3</sup> manually cataloged behavioral change problems (for some kinds of refactorings) in refactoring tools. Moreover, Max Schäfer and his colleagues explained an approach to solve problems on the rename refactoring.<sup>4</sup> We propose a more practical approach for detecting behavioral changes using tool support, regardless of the kind of refactoring.

### References

1. B. Daniel et al., "Automated Testing of Refactoring Engines," *Foundations of Software Eng.*, ACM Press, 2007, pp. 185–194.
2. F. Steimann and A. Thies, "From Public to Private to Absent: Refactoring Java Programs under Constrained Accessibility," *Proc. 23rd European Conf. Object-Oriented Programming (ECOOP 09)*, LNCS 5653, Springer, 2009, pp. 419–443.
3. T. Ekman et al., "Refactoring Bugs in Eclipse, Idea and Visual Studio," *Programming Tools Group*, 2008; <http://progtools.comlab.ox.ac.uk/projects/refactoring/bugreports>.
4. M. Schäfer, T. Ekman, and O. Moor, "Sound and Extensible Renaming for Java," *Proc. 23rd Conf. Object-Oriented Programming Systems Languages and Applications (OOPSLA 08)*, ACM Press, 2008, pp. 277–294.

|                                                                                                                                                                                                                         |                                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>public class A {     public int k(long i) {         return 10;     } }  public class B extends A {     public int k(int i) {         return 20;     } }  public int test() {     return new A().k(2); }  (a)</pre> | <pre>public class A {     public int k(long i) {         return 10;     }     public int k(int i) {         return 20;     } }  public class B extends A {     public int test() {         return new A().k(2);     } }  (b)</pre> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Figure 1. Pull-up method refactoring enables overloading: (a) source program and (b) target program. Eclipse doesn't detect a behavioral change in the test method.**

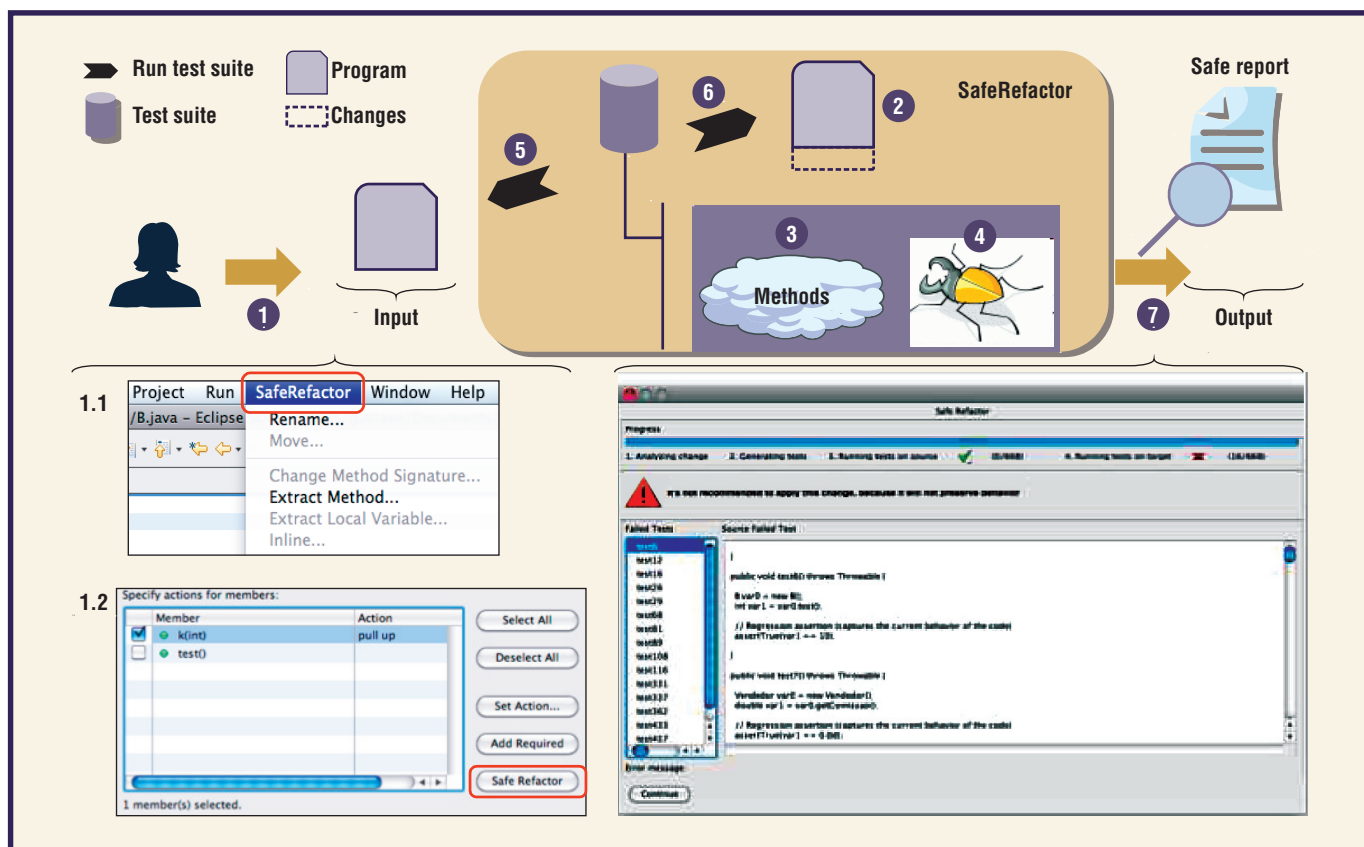
If it's called twice (source and target programs) with the same set of inputs, the resulting set of output values must be the same.<sup>6</sup> SafeRefactor checks the observable behavior with respect to randomly generated sequences of methods and constructor invocations. They only contain calls to methods common to both. If the source and target programs have different results for the same input, they don't have the same behavior.

### Evaluation

We evaluated SafeRefactor in 24 transformations.

### Subject Characterization

Tables 1 and 2 show the subjects (pairs of source



**Figure 2. SafeRefactor. Step 1: The developer (1.1) selects a refactoring in the menu to apply and (1.2) clicks on the SafeRefactor button. Step 2: The tool generates the target program using the Eclipse refactoring API and then (step 3) identifies common methods in the source and target programs. Step 4: The tool generates unit tests using the modified Randoop and then (step 5) runs the test suite on the source program. Step 6: The tool runs the test suite on the target program and then (step 7) shows the report to developer. If it finds a behavior change, the developer can see some unit tests that fail.**

and target programs) used in the experiment. Each of them is uniquely identified in the Subject column. We divided subjects into two categories: refactoring real applications and a catalog of defective refactorings (see [www.dsc.ufcg.edu.br/~spg/saferefactor/experiments.htm](http://www.dsc.ufcg.edu.br/~spg/saferefactor/experiments.htm)).

The first category (subjects 1 through 7) consists of the refactorings proposed in the literature applied to real Java applications ranging from 3 to 100 KLOC (nonblank, noncomment LOC) using tools or manual steps. Developers consider all of them “behavior preserving.” We then used SafeRefactor to evaluate whether the transformation preserves the observable behavior.

Third-party developers performed a refactoring on JHotDraw and CheckStylePlugin (subjects 1 and 2) to modularize exception-handling code.<sup>7</sup> Robert Fuhrer and his colleagues proposed and implemented an Eclipse refactoring to apply the infer generic type argument refactoring, letting applications use Java generics.<sup>8</sup> They evaluated their refactoring in four real Java applications:

JUnit, Vpoker, ANTLR, and Xtc (subjects 3 through 6). Emerson Murphy-Hill and his colleagues performed some experiments to analyze how developers refactor.<sup>9</sup> Specifically, they used a set of 20 Eclipse component versions from the Eclipse Concurrent Versions System (CVS) and manually detected the refactorings applied. We evaluated one transformation in the Eclipse TextEditor module (subject 7).

The second category (subjects 8 through 24) includes nonbehavior-preserving transformations applied by refactoring tools.<sup>3,4,10</sup> We used SafeRefactor to evaluate whether SafeRefactor detected the behavior changes.

## Experimental Setup

As mentioned earlier, we ran our experiment on a dual-processor 2.2-GHz Dell Vostro 1400 laptop with 2 Gbytes RAM and running Ubuntu 9.04. In both categories, we used a command-line interface provided by SafeRefactor that receives three parameters: source and target program paths,

**Table 1****Summary of SafeRefactor evaluation in refactoring real applications**

| Subject | Program          | KLOC | Refactoring               | Tests  | Error | Total time (sec) | Result            |
|---------|------------------|------|---------------------------|--------|-------|------------------|-------------------|
| 1       | JHotDraw         | 23   | Extract exception handler | 2,245  | 273   | 148              | Behavior change   |
| 2       | CheckStylePlugin | 20   | Extract exception handler | 5,864  | 0     | 235              | -                 |
| 3       | JUnit            | 3    | Infer generic type        | 1,127  | 0     | 99               | -                 |
| 4       | Vpoker           | 4    | Infer generic type        | 466    | 0     | 109              | -                 |
| 5       | ANTLR            | 32   | Infer generic type        | -      | -     | 2                | Compilation error |
| 6       | Xtc              | 100  | Infer generic type        | -      | -     | 4                | Compilation error |
| 7       | TextEditor       | 15   | Replace deprecated code   | 16,009 | 0     | 107              | -                 |

and timeout to generate tests. We used the default timeout of 90 seconds and 2 seconds to generate the tests in the first and second categories, respectively. We didn't use the SafeRefactor Eclipse graphical interface to automate the experiment.

### Experimental Results

SafeRefactor detected a behavioral change in one refactoring and two compilation errors in less than four minutes in the first category. Table 1 shows the program name, its size in KLOC, the refactoring applied, and the total time in seconds that SafeRefactor required to yield a result in the Program, KLOC, Refactoring, and Total time columns, respectively.

Each line of Tables 1 and 2 contains the number of generated tests (Tests column) and the number of tests detecting the behavioral change (Error column) for each subject. The Result column indicates whether SafeRefactor identified a behavior change or a compilation error. The symbol “-” indicates that it didn't detect a behavior change. In the second category, SafeRefactor detected all but one behavior change in less than eight seconds; Table 2 indicates the defective refactoring applied and a description of the behavior change introduced in the Refactoring and Bug description columns, respectively.

Developers refactored JHotDraw to avoid code duplication with identical exception handlers in different parts of a system.<sup>11</sup> Eight programmers working in pairs performed the change: they extracted the code inside the `try`, `catch`, and `finally` blocks to methods in specific classes that handle exceptions. They used refactoring tools, pair review, and unit tests to assure that the behavior was preserved. Some classes that implement `Serializable` were refactored. Devel-

opers changed the `done` method and introduced the `handler` attribute to handle exceptions. However, they forgot to serialize this new attribute. Thus, when the method `done` tried to serialize the object, an exception was thrown, meaning refactored method `done` has a different behavior. Moreover, Eclipse wrongly applied a refactoring to ANTLR and Xtc, introducing a compilation error that wasn't reported.<sup>8</sup>

SafeRefactor didn't detect behavior change in subjects 2 through 4 or 7. In the second category, SafeRefactor identified all but one behavior change that uses standard output. Tables 1 and 2 summarize the results.

**B**ecause our tool can't detect behavioral changes in the standard output (`System.out.println`) messages and exception messages (subject 8), we have to modify some subjects to include `return` statements with the values of the messages. We intend to generate some test patterns that are useful for detecting changes in the standard output. The current implementation of our technique can't deal with inner and nonpublic classes directly, but if there's a method in common using them, we can exercise them indirectly.

Nevertheless, SafeRefactor can detect behavior changes in rename refactoring. Consider a rename method from `A.m(..)` to `A.n(..)`. The set of methods that step 3 identifies doesn't include them. A similar situation occurs when renaming a class. We can't compare the renamed method's behavior directly, but SafeRefactor can compare them indirectly if a method in common ( $x$ ) calls them. Step 4 generates tests including  $x$  in the randomly generated sequence of method calls:

**Table 2****Summary of SafeRefactor evaluation in the catalog of defective refactorings**

| Subject | Refactoring             | Bug description                                                      | Tests | Error | Result          |
|---------|-------------------------|----------------------------------------------------------------------|-------|-------|-----------------|
| 8       | Push down method        | Incorrect handling of super accesses                                 | 488   | 0     | -               |
| 9       | Rename class            | Renaming a class leads to undiagnosed shadowing                      | 102   | 95    | Behavior change |
| 10      | Rename variable         | Renaming a local variable leads to shadowing by field                | 494   | 492   | Behavior change |
| 11      | Rename method           | Renaming a method leads to shadowing of statically imported method   | 93    | 91    | Behavior change |
| 12      | Encapsulate field       | No check for overriding problems                                     | 474   | 464   | Behavior change |
| 13      | Extract method          | Incorrect dataflow analysis                                          | 558   | 554   | Behavior change |
| 14      | Push down method        | Incorrect handling of super accesses                                 | 486   | 404   | Behavior change |
| 15      | Push down method        | Incorrect handling of field accesses                                 | 78    | 75    | Behavior change |
| 16      | Push down method        | Pushing down a method enables overloading                            | 101   | 99    | Behavior change |
| 17      | Move class              | Move a class to another package disables overriding                  | 101   | 99    | Behavior change |
| 18      | Move class              | Move a class to another package disables overloading                 | 79    | 77    | Behavior change |
| 19      | Change method signature | Increasing method visibility enables overriding                      | 214   | 40    | Behavior change |
| 20      | Change method signature | Increasing method visibility enables overloading                     | 79    | 76    | Behavior change |
| 21      | Change method signature | Increase method visibility enables overriding to another package     | 121   | 40    | Behavior change |
| 22      | Pull up method          | Pulling up a method enables overloading                              | 101   | 99    | Behavior change |
| 23      | Pull up method          | Pulling up a method enables overriding                               | 170   | 88    | Behavior change |
| 24      | Pull up method          | Pulling up a method to a class in another package enables overriding | 167   | 163   | Behavior change |

if a method calls them, we can compare them indirectly. However, a simple rename method could enable or disable overloading,<sup>10</sup> a feature that offers a potential source of problems. Step 4 generates specific tests for exercising every method (not affected by the transformation) named *m* or *n* in the superclasses or subclasses of *A*. Table 2 shows three defective renaming refactorings that our tool detected.

Our earlier work presents our technique<sup>12</sup> and a tool in more detail.<sup>13</sup> Here, we evaluated our technique with empirical studies. In the near future, we intend to create a plugin for other IDEs, such as Netbeans. In addition to using it in more real case studies, we plan to test refactoring tools to automatically find nonbehavior-preserving transformations following a similar approach to that of Brett Daniel and his colleagues.<sup>14</sup> 

### Acknowledgments

We thank the guest editors and the anonymous referees for useful suggestions. This work was partial-

ly supported by CNPq grants 573964/2008-4 and 477336/2009-4.

### References

1. M. Fowler, *Refactoring: Improving the Design of Existing Code*, Addison-Wesley, 1999.
2. T. Mens and T. Tourwé, "A Survey of Software Refactoring," *IEEE Trans. Software Eng.*, vol. 30, no. 2, 2004, pp. 126–139.
3. F. Steimann and A. Thies, "From Public to Private to Absent: Refactoring Java Programs under Constrained Accessibility," *Proc. 23rd European Conf. Object-Oriented Programming (ECOOP 09)*, LNCS 5653, Springer, 2009, pp. 419–443.
4. T. Ekman et al., "Refactoring Bugs in Eclipse, Idea and Visual Studio," *Programming Tools Group*, 2008; <http://progtools.comlab.ox.ac.uk/projects/refactoring/bugreports>.
5. C. Pacheco et al., "Feedback-Directed Random Test Generation," *Proc. 29th Int'l Conf. Software Eng. (ICSE 07)*, IEEE CS Press, 2007, pp. 75–84.
6. W.F. Opdyke, *Refactoring Object-Oriented Frameworks*, tech. report UIUCDCS-R-92-1759, Univ. of Illinois at Urbana-Champaign, 1992.
7. J. Taveira et al., "Assessing Intra-Application Exception Handling Reuse with Aspects," *Proc. 23rd Brazilian Symp. Software Eng. (SBES 09)*, IEEE CS Press, 2009, pp. 22–31.



8. R. Fuhrer et al., "Efficiently Refactoring Java Applications to Use Generic Libraries," *Proc. 19th European Conf. Object-Oriented Programming (ECOOP 05)*, LNCS 3586, Springer, 2005, pp. 71–96.
9. E. Murphy-Hill, C. Parnin, and A. Black, "How We Refactor, and How We Know It," *Proc. 31st Int'l Conf. Software Eng. (ICSE 09)*, IEEE CS Press, 2009, pp. 287–296.
10. M. Schäfer, T. Ekman, and O. Moor, "Sound and Extensible Renaming for Java," *Proc. 23rd Conf. Object-Oriented Programming Systems Languages and Applications (OOPSLA 08)*, ACM Press, 2008, pp. 277–294.
11. B. Cabral and P. Marques, "Exception Handling: A Field Study in Java and .Net," *Proc. 21st European Conf. Object-Oriented Programming (ECOOP 07)*, LNCS 4609, Springer, 2007, pp. 151–175.
12. G. Soares et al., "Generating Unit Tests for Checking Refactoring Safety," *Proc. Brazilian Symp. Programming Languages (SBPL 09)*, Brazilian Computer Society, 2009, pp. 159–172.
13. G. Soares et al., "SafeRefactor—Tool for Checking Refactoring Safety," *Proc. Brazilian Symp. Software Engineering (SBES 09)*, Brazilian Computer Society, 2009, pp. 49–54.
14. B. Daniel et al., "Automated Testing of Refactoring Engines," *Proc. ACM SIGSOFT Symp. Foundations of Software Engineering (FSE 07)*, ACM Press, 2007, pp. 185–194.

## About the Authors



**Gustavo Soares** is a PhD student in the Department of Computer Science at the Federal University of Campina Grande. His research interests include refactoring and formal methods. Soares has an MSc in computer science from the Federal University of Campina Grande. He's a member of IEEE and the ACM. Contact him [gsoares@dsc.ufcg.edu.br](mailto:gsoares@dsc.ufcg.edu.br).

**Rohit Gheyi** is a professor in the Department of Computer Science at the Federal University of Campina Grande. His research interests include refactoring, formal methods, and software product lines. Gheyi has a PhD in computer science from the Federal University of Pernambuco. He's a member of the ACM. Contact him at [rohit@dsc.ufcg.edu.br](mailto:rohit@dsc.ufcg.edu.br).



**Dalton Serey** is a professor in the Department of Computer Science at the Federal University of Campina Grande. His research interests include software development, evolution, and verification. Serey has a PhD in electrical engineering from the Federal University of Paraíba. He's a member of the ACM. Contact him [dalton@dsc.ufcg.edu.br](mailto:dalton@dsc.ufcg.edu.br).

**Tiago Massoni** is a professor in the Department of Computer and Systems at the Federal University of Campina Grande. His research interests include software design and evolution, and formal methods. Massoni has a PhD in computer science from the Federal University of Pernambuco. He's a member of the ACM. Contact him at [massoni@dsc.ufcg.edu.br](mailto:massoni@dsc.ufcg.edu.br).



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.

## ADVERTISER INFORMATION

### JULY/AUGUST 2010 • IEEE SOFTWARE

#### Advertiser

Seapine Software Cover 4  
The University of Chicago Press 11

#### Advertising Personnel

Marion Delaney  
IEEE Media, Advertising Dir.  
Phone: +1 415 863 4717  
Email: [md.ieeemedia@ieee.org](mailto:md.ieeemedia@ieee.org)

Marian Anderson  
Sr. Advertising Coordinator  
Phone: +1 714 821 8380  
Fax: +1 714 821 4010  
Email: [manderson@computer.org](mailto:manderson@computer.org)

Sandy Brown  
Sr. Business Development Mgr.  
Phone: +1 714 821 8380  
Fax: +1 714 821 4010  
Email: [sb.ieeemedia@ieee.org](mailto:sb.ieeemedia@ieee.org)

#### Advertising Sales Representatives

##### Recruitment:

Mid Atlantic  
Lisa Rinaldo  
Phone: +1 732 772 0160  
Fax: +1 732 772 0164  
Email: [lr.ieeemedia@ieee.org](mailto:lr.ieeemedia@ieee.org)

New England  
John Restchack  
Phone: +1 212 419 7578  
Fax: +1 212 419 7589  
Email: [j.restchack@ieee.org](mailto:j.restchack@ieee.org)

Southeast  
Thomas M. Flynn  
Phone: +1 770 645 2944  
Fax: +1 770 993 4423  
Email: [flynnntom@mindspring.com](mailto:flynnntom@mindspring.com)

Midwest/Southwest  
Darcy Giovino  
Phone: +1 847 498 4520  
Fax: +1 847 498 5911  
Email: [dg.ieeemedia@ieee.org](mailto:dg.ieeemedia@ieee.org)

Northwest/Southern CA  
Tim Matteson  
Phone: +1 310 836 4064  
Fax: +1 310 836 4067  
Email: [tm.ieeemedia@ieee.org](mailto:tm.ieeemedia@ieee.org)

Japan  
Tim Matteson  
Phone: +1 310 836 4064  
Fax: +1 310 836 4067  
Email: [tm.ieeemedia@ieee.org](mailto:tm.ieeemedia@ieee.org)

Europe  
Heleen Vodegel  
Phone: +44 1875 825700  
Fax: +44 1875 825701  
Email: [impress@impressmedia.com](mailto:impress@impressmedia.com)

#### Product:

US East  
Dawn Becker  
Phone: +1 732 772 0160  
Fax: +1 732 772 0164  
Email: [db.ieeemedia@ieee.org](mailto:db.ieeemedia@ieee.org)

US Central  
Darcy Giovino  
Phone: +1 847 498 4520  
Fax: +1 847 498 5911  
Email: [dg.ieeemedia@ieee.org](mailto:dg.ieeemedia@ieee.org)

US West  
Lynne Stickrod  
Phone: +1 415 931 9782  
Fax: +1 415 931 9782  
Email: [ls.ieeemedia@ieee.org](mailto:ls.ieeemedia@ieee.org)

Europe  
Sven Anacker  
Phone: +49 202 27169 11  
Fax: +49 202 27169 20  
Email: [sanacker@intermediapartners.de](mailto:sanacker@intermediapartners.de)